Google TimesFM Implementation Handbook

Version: `v1.0.0` (planned release)

This handbook is the operational and technical manual for the repository at: `/home/ahmad/AI/Github/google-TimesFM-implementation`

It is written from actual repository code and real generated outputs. No synthetic or mocked metrics are reported.

---

1) Glossary and Definitions

- **TimesFM**: Decoder-only foundation model for time-series forecasting by Google Research.
- **Horizon**: Forecast length ahead (for example, 24 hours, 7 days, 30 days).
- **Backtest**: Historical rolling evaluation to estimate expected forecasting error.
- **MAE**: Mean absolute error.
- **RMSE**: Root mean squared error.
- **WMAPE**: Weighted mean absolute percentage error.
- **NWRMSLE**: Normalized weighted root mean squared logarithmic error (used in retail validation outputs).
- **Series panel**: Data layout where each row represents one time series and contains sequence values.
- **XReg**: External regressors/covariates mode used with TimesFM in retail workflow.
- **MILP**: Mixed-integer linear programming; used for warehouse allocation optimization.
- **Run ID**: Immutable execution identifier for final real runs (example: `real_20260703_final_v6`).

---

2) Project Purpose

The project operationalizes TimesFM across 10 business-critical forecasting use cases and converts forecasts into planning outputs.

Target business outcomes

- Inventory optimization (retail)
- Grid stability and cost control (electricity)
- Predictive maintenance (manufacturing)
- Staffing and bed planning (hospital)
- Cash logistics optimization (ATM)
- Auto-scaling and cost reduction (cloud + web)

- Revenue and capacity planning (airline + transactions + warehouse)

---

3) Prerequisites

Runtime

- Linux shell
- Python `>=3.11`
- `uv` package manager
- Kaggle credentials for Kaggle-based data pulls

System tools used in this repo

- `7z` for `.7z` archive extraction (Favorita files)
- Jupyter/nbconvert for notebook execution
- Pandoc + LaTeX engine for PDF generation (`xelatex` or `pdflatex`)

---

4) Environment Setup

```
uv venv
source .venv/bin/activate
uv sync
```

Optional kernel setup:

```
python -m ipykernel install --user --name timesfm-local --display-name "Python (timesfm-loca
```

Kaggle configuration check:

```
kaggle config view
```

Observed local config during documentation work: - `username: pypiahmad` - `auth_method: ACCESS_TOKEN`

---

5) Dependency Explanation

Defined in `pyproject.toml`.

Primary categories: - **Modeling**: `timesfm[torch,xreg]` - **Data processing**: `pandas`, `polars`, `pyarrow`, `scikit-learn` - **Optimization**: `ortools` - **Validation and config**: `pydantic` - **Logging/progress**: `loguru`, `tqdm` - **Execution**: `jupyter`, `nbconvert`, `ipykernel` - **Data access**: `kaggle`

---

6) Folder Structure

High-level structure:

```
.
+-- artifacts/
|   +-- final_real_runs/
|   |   +-- real_20260703_final_v6/
|   +-- <domain-specific outputs>
+-- data/
|   +-- favorita_real/
|   +-- electricity_load/
|   +-- manufacturing/
|   +-- ...
+-- notebooks/
|   +-- <10 source notebooks>
|   +-- <10 executed notebooks>
|   +-- final_real_runs/
+-- scripts/
|   +-- retail_demand_timesfm_favorita.py
|   +-- final_real_run.py
|   +-- validate_final_real_run.py
|   +-- build_docs_evidence.py
+-- README.md
+-- HANDBOOK.md
+-- pyproject.toml
```

Notes: - `artifacts/final_real_runs/real_20260703_final_v6` is the authoritative validated output set used in this handbook. - `data/favorita_real/raw/train.csv` is large (>4.9 GB), which impacts repository publishing strategy.

---

7) Code Walkthrough

7.1 `scripts/retail_demand_timesfm_favorita.py`

Key behaviors: - Defines `PipelineConfig` via Pydantic. - Downloads Favorita competition archives via Kaggle CLI and extracts with `7z`. - Builds filtered train-long and panel parquet datasets. - Runs TimesFM forecasting for horizons (`7/14/30`; plus generated horizon directories in outputs). - Computes validation metrics and exports: - `validation_metrics.csv` - `inventory_policy.parquet` - `warehouse_allocation_heuristic.parquet` - `warehouse_allocation_milp.parquet` - `submission_timesfm.csv` (when enabled) - Includes OR-Tools MILP option for warehouse allocation.

7.2 `scripts/final_real_run.py`

Purpose: - Decision-complete orchestration across all notebooks and script targets. - Patches notebook constants (project root and artifact paths) to isolated run directories. - Executes notebooks with `jupyter nbconvert --execute`. - Supports retail reuse mode with: - `--reuse-retail-artifacts-from` - `--reuse-retail-executed-notebook` - Writes run report JSON with commands, durations, status, log paths.

7.3 `scripts/validate_final_real_run.py`

Purpose: - Verifies run integrity: - executed notebooks exist and have no error outputs - required artifacts exist per domain - strict real checks for large retail data and non-trivial horizon row counts - Writes `final_validation_report.json`.

7.4 `scripts/build_docs_evidence.py`

Purpose: - Converts run and validation JSON + per-domain metrics into documentation-safe evidence: - `docs/evidence/real_20260703_final_v6/summary.json` - `docs/evidence/real_20260703_final_v6/summary.md`

---

8) Training / Inference / Pipeline Flow

End-to-end operational flow

1. Acquire data from web sources.
2. Build domain-specific clean series.
3. Run TimesFM inference (and baselines for comparison).
4. Evaluate via backtesting metrics.
5. Produce decision outputs.
6. Record run metadata and logs.
7. Validate output completeness and integrity.

Domain output mapping

Problem	Forecast Target	Output Examples
Retail demand	Product sales	inventory policy, warehouse allocation, submission
Electricity load	Power demand	weekly operational schedule
Manufacturing sensors	Future temperature	alerts + maintenance schedule

Problem	Forecast Target	Output Examples
Hospital patients	Patient arrivals	staffing and holiday staffing plans
ATM cash	Cash withdrawals	refill plan + logistics summary
Cloud capacity	CPU utilization	autoscaling KPIs + 24h capacity plan
Airline demand	Passenger counts	pricing signals + fleet/crew planning
Warehouse orders	Daily orders	warehouse operations plan
Website traffic	Hourly traffic	autoscaling plan + infra KPI summary
Payment transactions	Txn volume	capacity resource plan

---

9) Commands Used

9.1 Core execution commands (from real run report)

Representative commands executed by orchestrator:

```
/home/ahmad/AI/Github/google-TimesFM-implementation/.venv/bin/python -m jupyter nbconvert --
```

Retail targets in run `real_20260703_final_v6` executed in reuse mode: - notebook target: `reuse_retail_artifacts` - script target: `reuse_retail_artifacts`

9.2 Validation command

```
.venv/bin/python scripts/validate_final_real_run.py --run-id real_20260703_final_v6 --strict
```

9.3 Documentation evidence command

```
.venv/bin/python scripts/build_docs_evidence.py --run-id real_20260703_final_v6
```

9.4 Handbook PDF command

```
pandoc HANDBOOK.md -o HANDBOOK.pdf --pdf-engine=xelatex
```

---

10) Configuration Explanation

Retail pipeline arguments (`retail_demand_timesfm_favorita.py`)

Important knobs: - `--data-root` - `--artifacts-root` - `--horizons` - `--context-len` - `--per-core-batch-size` - `--forecast-batch-size` -

`--device {auto,cpu,cuda}` - `--use-xreg`, `--xreg-mode`, `--xreg-ridge` - `--run-milp`, `--n-warehouses`, `--warehouse-capacity-factor` - `--service-level`, `--lead-time-days` - `--max-series`, `--validation-series-limit` - `--include-submission`

Orchestrator arguments (`final_real_run.py`)

- `--run-id`
- `--include-notebooks / --exclude-notebooks`
- `--include-scripts / --exclude-scripts`
- `--fail-fast / --no-fail-fast`
- `--reuse-retail-artifacts-from`
- `--reuse-retail-executed-notebook`

Validator arguments (`validate_final_real_run.py`)

- `--run-id`
- `--strict-real-checks`
- `--output-report`

---

11) Validation, Evaluation, and Metrics

Authoritative files: - `artifacts/final_real_runs/real_20260703_final_v6/final_run_report.json` - `artifacts/final_real_runs/real_20260703_final_v6/final_validation_report.json` - `docs/evidence/real_20260703_final_v6/summary.json`

Run-level status: - Run summary: `PASS` (`11/11` successful targets) - Validation summary: `PASS` (`98/98` checks)

Domain metric snapshot (mean metric by model across recorded horizons in artifact CSVs):

Domain	Metric	TimesFM	Best Baseline	Best Overall
Airline	MAE	20.2280	22.8679	TimesFM
ATM cash	MAE	3141.9153	4215.2616	TimesFM
Cloud capacity	MAE	2.6500	4.2666	TimesFM
Electricity load	MAE	1046.8886	1413.8467	TimesFM
Hospital	MAE	534.2010	740.0851	TimesFM
Manufacturing	MAE	2.6583	3.4890	TimesFM
Transactions	MAE	5.2125	7.6087	TimesFM
Warehouse orders	MAE	46.7143	56.4423	TimesFM

Domain	Metric	TimesFM	Best Baseline	Best Overall
Website traffic	MAE	354.3039	577.5542	TimesFM
Retail (notebook)	NWRMSLE	1.9480	1.1561	Baseline (`naive_seasonal7`)
Retail (script)	NWRMSLE	1.9480	1.1561	Baseline (`naive_seasonal7`)

Interpretation: - TimesFM is strongest on the non-retail domains in this validated run. - Retail requires further calibration/feature engineering for this specific setup.

---

12) Outputs and Interpretation

Each domain writes artifacts in CSV/parquet form that map to operational decisions.

Examples: - `autoscaling_plan.csv` (website traffic): projected scaling actions. - `capacity_resource_plan.csv` (transactions): capacity/fraud-monitoring planning. - `staffing_plan_next_14_days.csv` (hospital): staffing recommendations. - `fleet_allocation_plan.csv` (airline): route-level fleet guidance. - `inventory_policy.parquet` (retail): reorder strategy inputs.

Execution logs are in: - `artifacts/final_real_runs/real_20260703_final_v6/logs/`

---

13) Debugging and Troubleshooting

Common failure points

1. Kaggle data download errors (`403 Forbidden` or auth failures)

- Cause: competition rules not accepted or invalid credentials.
- Fix: accept competition terms on Kaggle and verify `kaggle config view`.

2. Network/DNS failures for Kaggle API

- Cause: restricted network environment.
- Fix: run with outbound network access and retry downloads.

3. Notebook execution timeout or resource pressure

- Cause: large data + constrained hardware.
- Fix: run with CPU-safe settings or adjust series limits where supported.

4. Missing retail artifacts during reuse mode

- Cause: invalid path passed to `--reuse-retail-artifacts-from`.
- Fix: point to a valid directory containing expected horizon outputs.

Diagnostic files

- `final_run_report.json` for command-level return codes and durations.
- `final_validation_report.json` for artifact-level checks.
- per-target logs in run `logs/` directory.

---

14) Deployment or Execution Notes

This repository is currently organized for reproducible batch workflows, not online serving.

For productionization: - Wrap selected domain pipelines in scheduled jobs. - Persist model/forecast metadata per run in a registry. - Add drift/error monitoring and alerting. - Standardize output schema contracts consumed by downstream systems.

---

15) Best Practices and Lessons Learned

- Keep run outputs isolated by run ID for reproducibility and rollback.
- Validate every run with strict artifact checks before claiming success.
- Separate raw data acquisition, model inference, and decision layers.
- Track cases where baselines beat TimesFM (retail here) and treat as optimization backlog, not hidden failure.
- For heavy datasets/artifacts, publish code and lightweight evidence in GitHub; publish full snapshots via dataset hosting (Kaggle recommended in this project).

---

16) References / Sources

Core model

- Google TimesFM repository: https://github.com/google-research/timesfm
- Google TimesFM research blog: https://research.google/blog/a-decoder-only-foundation-model-for-time-series-forecasting/
- Hugging Face model card: https://huggingface.co/google/timesfm-2.5-200m-pytorch

APIs and frameworks

- Kaggle API docs: https://www.kaggle.com/docs/api

- OR-Tools docs: https://developers.google.com/optimization
- Polars docs: https://pola.rs/
- PyArrow docs: https://arrow.apache.org/docs/python/
- Pydantic docs: https://docs.pydantic.dev/

Dataset sources referenced in notebooks/scripts

- Favorita competition: https://www.kaggle.com/competitions/favorita-grocery-sales-forecasting
- ATM dataset: https://www.kaggle.com/datasets/zoya77/atm-cash-demand-forecasting-and-management
- Airline dataset: https://www.kaggle.com/datasets/saadharoon27/airlines-dataset
- Hourly energy dataset: https://www.kaggle.com/datasets/robikscube/hourly-energy-consumption
- Manufacturing dataset: https://www.kaggle.com/datasets/anshtanwar/metro-train-dataset
- Olist e-commerce dataset: https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce
- NASA logs dataset: https://www.kaggle.com/datasets/pasanbhanuguruge/nasa-http-logs-dataset-processed-for-lstm-models
- Financial transactions dataset: https://www.kaggle.com/datasets/sergionefedov/fraud-detection-1m-transactions-7-fraud-types
- HHS hospital dataset: https://healthdata.gov/Hospital/COVID-19-Reported-Patient-Impact-and-Hospital-Capa/g62h-syeh
- NAB repository: https://github.com/numenta/NAB

---

17) Evidence Appendix

Primary evidence files generated for documentation: - `docs/evidence/real_20260703_final_v6/summary.md`
- `docs/evidence/real_20260703_final_v6/summary.json`

These are derived directly from: - `artifacts/final_real_runs/real_20260703_final_v6/final_run_repor`
- `artifacts/final_real_runs/real_20260703_final_v6/final_validation_report.json`
- domain metrics CSVs under the same run directory.